

MangaApp: подробное объяснение архитектуры, данных и источников

Подробное объяснение текущего мобильного приложения, источников данных, Reader, SQLite, TanStack Query и будущего добавления сайтов.

Версия документа: 2026-06-29

Этот документ объясняет текущее состояние приложения MangaApp так, чтобы ты мог не просто "знать где что лежит", а понимать мышление проекта: почему данные идут именно так, как добавляются сайты, почему некоторые сайты легко подключить, почему другие ломаются, где живет прогресс чтения, как работают языки, источники, поиск, Reader, Library и "Also available on".

Важно: README в репозитории частично устарел. Он все еще описывает старую идею, где app говорит только с backend. Текущее приложение в папке app/src уже в основном работает иначе: источники запускаются прямо на устройстве через `src/data/sources/providers`. Backend все еще есть в папке `backend/`, но сейчас он не является главным путем получения данных для мобильного приложения.

1. Что это за приложение

MangaApp - мобильное приложение для Android на Expo/React Native. Его идея: читать мангу, манхву, вебтуны и похожий контент из разных источников, но показывать все через единый UI.

Пользователь видит:

- Home - топ/новинки/продолжить чтение.
- Explore - поиск по одному источнику или сразу по всем включенным источникам.
- Sources - выбор языка и источника.
- Manga details - страница тайтла, описание, обложка, главы, переключение источника и языка.
- Reader - чтение страниц, вертикально или постранично.
- Library - сохраненные тайтлы, прогресс, статусы.
- Updates - новые главы по тайтлам, которые пользователь уже начал читать.
- Settings - языки, скрывание источников, настройки ридера, очистка данных.
- Diagnostics - проверка источников.

Ключевая идея: UI не должен знать, как устроен MangaDex, MangaLib, WEBTOON или MangaKatana. UI получает нормализованные объекты одного формата. Вся грязь сайтов спрятана внутри provider-классов.

2. Какие языки программирования и технологии используются

Основной язык проекта - TypeScript. Он используется и в мобильном приложении, и в backend-папке.

Используемые языки/форматы:

- TypeScript - логика приложения, провайдеры источников, React components, hooks, backend.

- TSX/JSX - экраны и компоненты React Native.
- JavaScript runtime - приложение выполняется в React Native/Expo окружении.
- SQL - локальная база SQLite через `expo-sqlite`.
- JSON - ответы API источников, настройки Expo, package.json, app.json.
- CSS-like React Native StyleSheet - стили экранов и компонентов.

Главные библиотеки и фреймворки:

- Expo SDK 56 - сборка и инфраструктура React Native app.
- React Native 0.85 - мобильный UI.
- React 19 - компонентная модель.
- Expo Router - file-based routing: папка `app/` становится навигацией.
- TanStack Query - сетевые запросы, cache, refetch, состояние загрузки/ошибок.
- TanStack Query Persist Client + AsyncStorage - сохранение части query-cache между запусками.
- Zustand - глобальные пользовательские настройки.
- Expo SQLite - локальная база для прогресса, библиотеки, linked works.
- Expo Image - показ обложек и страниц с cache.
- FlashList - быстрый список страниц в Reader.
- React Native Gesture Handler + Reanimated - zoom, gestures, плавность.
- Expo Keep Awake - не гасить экран во время чтения.
- Ionicons - иконки.

3. Главная архитектурная идея

Приложение делится на слои:

1. UI слой: `app/` и `src/components/`
2. Query слой: `src/data/queries.ts`
3. Source слой: `src/data/sources/`
4. Local DB слой: `src/data/local/db.ts`
5. Store слой: `src/store/`
6. Theme/lib слой: `src/theme/`, `src/lib/`
7. Legacy/backend слой: `backend/`

Схема:

Screen

- > query hook
- > SourceManager.require(sourceId)
- > provider method
- > fetchWithTimeout(url)
- > external site/API

Screen

- > local query/mutation
- > db.ts
- > expo-sqlite

Например, Home хочет показать popular MangaDex:

```
HomeScreen
-> useTrending('mangadex', 'en', 'popular')
-> SourceManager.require('mangadex').trending({ lang: 'en' })
-> MangaDexProvider.trending()
-> fetch https://api.mangadex.org/manga?...
-> map raw MangaDex response to MangaSearchResult[]
```

UI не знает URL MangaDex. UI знает только, что есть `useTrending`.

4. Expo Router и экраны

Маршруты лежат в `app/`.

Главные файлы:

- `app/\_layout.tsx` - root layout, QueryClient, persisted cache, SafeArea, Stack navigation.
- `app/(tabs)/\_layout.tsx` - нижние Android tabs: Home, Updates, Explore, Sources, Library.
- `app/(tabs)/index.tsx` - Home.
- `app/(tabs)/explore.tsx` - поиск.
- `app/(tabs)/sources.tsx` - выбор источников.
- `app/(tabs)/updates.tsx` - обновления.
- `app/(tabs)/library.tsx` - библиотека.
- `app/manga/[id].tsx` - страница тайтла.
- `app/reader/[chapterId].tsx` - ридер.
- `app/settings.tsx` - настройки.
- `app/diagnostics.tsx` - диагностика источников.
- `app/top.tsx` - полный список top/latest.

Expo Router использует файловую структуру как routing. Если файл называется `app/manga/[id].tsx`, значит маршрут принимает dynamic param `id`. При открытии manga page приложение передает еще `sourceId`, потому что один и тот же `id` имеет смысл только внутри конкретного источника.

Пример перехода:

```
router.push({
  pathname: '/manga/[id]',
  params: { id: item.externalId, sourceId: item.sourceId },
});
```

5. Root layout: глобальный QueryClient и cache

Файл: `app/\_layout.tsx`

Там создается `QueryClient` для TanStack Query. Он отвечает за:

- хранение результатов запросов;
- состояние loading/error/success;
- retry;

- refetch;
- cache invalidation;
- persistence через AsyncStorage.

Важная настройка:

```

■■ persist:
- pages
- match
- sources

```

Почему:

- `pages` не сохраняются, потому что ссылки на картинки часто временные или подписанные.
- `match` не сохраняется, потому что matching может устареть, а источники меняются.
- `sources` не сохраняется, потому что список провайдеров локальный и дешевый; если persist его навсегда, новые источники могут не появиться.

Это хорошее решение. Оно показывает, что не всякий cache полезен. Иногда cache прячет новые данные или ломает свежесть.

6. SourceProvider - главный контракт всех сайтов

Файл: `src/data/sources/SourceProvider.ts`

Каждый источник должен реализовать один интерфейс:

```

export interface SourceProvider {
  id: string;
  name: string;
  languages: string[];
  type: 'official_api' | 'scraper' | 'user_files' | 'external_link';
  supportsSearch: boolean;
  supportsReading: boolean;

  trending(options?: SearchOptions): Promise<MangaSearchResult[]>;
  search(query: string, options?: SearchOptions): Promise<MangaSearchResult[]>;
  getMangaDetails(externalId: string): Promise<MangaDetails>;
  getChapters(externalId: string, lang?: string): Promise<Chapter[]>;
  getChapterPages(chapterId: string): Promise<ChapterPage[]>;
}

```

Это сердце приложения. Любой сайт - MangaDex, WEBTOON, MangaLib, MangaKatana - должен привести свои странные ответы к этому формату.

Что означает каждый метод:

`trending(options)`

Дает список популярных или новых тайтлов. Используется на Home и Top pages.

Важно: разные сайты понимают "popular" по-разному. У MangaDex это `followedCount`, у MangaKatana сейчас используется каталог latest, у Asura используется catalog page, потому что ranking page JS-only.

`search(query, options)`

Ищет тайтлы. Используется Explore и cross-source matching.

Для хорошего источника search должен:

- быстро отвечать;
- возвращать title, externalId, coverUrl, languages;
- желательно возвращать status и altTitles;
- не возвращать мусор без глав, если сайт позволяет отфильтровать.

``getMangaDetails(externalId)``

Загружает полную страницу тайтла: описание, авторы, жанры, год, статус, обложка.

Этот метод обычно вызывается при открытии manga page и во время более точного "Also available on" matching.

``getChapters(externalId, lang)``

Возвращает список глав. Это отдельный метод, потому что поиск может быть дешевым, а главы - дорогими.

Очень важный принцип: ``getChapters`` должен возвращать только readable chapters, то есть главы, которые реально можно открыть в приложении. Если глава ведет на внешнюю ссылку, платная, заблокирована или не дает страницы, лучше ее не возвращать.

``getChapterPages(chapterId)``

Возвращает страницы конкретной главы:

```
{
  index: number;
  imageUrl: string;
  headers?: Record<string, string>;
  width?: number;
  height?: number;
}
```

Headers нужны, потому что многие CDN не отдадут картинки без Referer.

7. Нормализованные типы данных

Файл: ``src/data/sources/types.ts``

``MangaSearchResult``

Минимальная карточка тайтла:

- ``sourceId`` - откуда тайтл.
- ``externalId`` - id/slug внутри сайта.
- ``title`` - название.
- ``altTitles`` - альтернативные названия.
- ``coverUrl`` - обложка.

- ``description`` - иногда есть уже в search.
- ``status`` - ongoing/completed/hiatus/unknown.
- ``languages`` - языки, на которых источник/тайтл доступен.

``MangaDetails``

Расширяет search result:

- ``authors``
- ``genres``
- ``year``
- ``contentRating``

``Chapter``

Глава:

- ``sourceId``
- ``externalId``
- ``mangaExternalId``
- ``title``
- ``chapterNumber``
- ``volume``
- ``language``
- ``publishedAt``
- ``scanlationGroup``

``ChapterPage``

Страница:

- ``index``
- ``imageUrl``
- ``width``
- ``height``
- ``headers``
- ``expiresAt``

Идея нормализации: UI работает с этими типами и не зависит от конкретного сайта.

8. Registry и SourceManager

Файл: ``src/data/sources/registry.ts``

Сейчас зарегистрированы:

- MangaDex - multi-language official API.
- Mangapill - EN HTML scraper.
- WEBTOON - EN HTML + mobile API.
- Asura Scans - EN sitemap + HTML scraper.
- MangaKatana - EN HTML scraper.
- MangaLib - RU JSON API-like scraper.
- ReManga - RU JSON API-like scraper.
- MangaBuff - RU HTML + JSON suggestions.

`SourceRegistry.all()` возвращает все провайдеры.

`SourceRegistry.get(id)` возвращает provider по id.

`SourceManager.require(sourceId)` возвращает provider или кидает ошибку, если id неизвестен.

Почему это удобно:

- добавить источник = новый файл provider + одна строка в registry;
- UI не меняется;
- queries не меняются;
- Reader не меняется;
- Library не меняется.

9. Как приложение берет данные с сайтов

Есть несколько способов.

9.1 Official API

Пример: MangaDex.

MangaDex дает нормальный JSON API:

```
https://api.mangadex.org/manga
https://api.mangadex.org/manga/{id}/feed
https://api.mangadex.org/at-home/server/{chapterId}
```

Плюсы:

- стабильно;
- не надо парсить HTML;
- можно фильтровать язык;
- можно фильтровать наличие глав;
- понятные поля.

Минусы:

- есть rate limits;
- часть лицензированных глав может быть externalUrl и не читается в app.

9.2 JSON API, но не совсем официальный

Примеры: MangaLib, ReManga.

Они отдают JSON, но это не обязательно публичный официальный API. Приложение использует endpoint, который сайт использует сам.

Плюсы:

- намного легче HTML;
- есть meta, chapters, pages;
- часто есть image dimensions.

Минусы:

- endpoint может поменяться;
- могут требоваться headers;
- могут быть антибот/geo/rate limit.

9.3 HTML scraping

Примеры: Mangapill, MangaKatana, MangaBuff.

Приложение скачивает HTML страницу и регулярками вытаскивает:

- ссылки на тайтлы;
- title;
- cover;
- chapters;
- image URLs.

Пример:

```
GET https://mangakatana.com/manga/{slug}
parse <h1>
parse chapter links
parse reader script var thzq = [...]
```

Плюсы:

- работает без backend;
- часто достаточно для server-rendered сайтов;
- легко начать.

Минусы:

- если сайт поменяет HTML классы, парсер ломается;

- JS-only страницы плохо читаются;
- Cloudflare/DDoS защита может заблокировать fetch;
- regex-парсинг требует аккуратности.

9.4 Sitemap + local index

Пример: Asura.

У Asura поиск через JS-only API недоступен, поэтому provider делает хитрый ход:

- скачивает sitemap series;
- строит локальный список slug/title;
- ищет по этому списку на устройстве;
- details/chapters/pages берет из HTML.

Это очень полезный паттерн для сайтов, где search закрыт, но sitemap открыт.

9.5 SSR state / embedded data

Идея для будущего: некоторые сайты кладут готовые данные в HTML:

```
window.__SENKURO__ = {...}
__NEXT_DATA__ = {...}
window.__NUXT__ = {...}
```

Если нужные данные уже там, можно не реверсить весь API. Нужно:

1. скачать HTML;
2. найти embedded state;
3. распарсить JSON;
4. достать catalog/details/chapters/pages.

Это хороший метод для Next/Nuxt/SSR сайтов.

9.6 WebView/headless/proxy

Это крайняя мера для Cloudflare/DDoS-Guard.

Минусы:

- медленно;
- нестабильно;
- плохо для фонового поиска;
- сложно хостить;
- может ломаться после обновлений защиты;
- увеличивает стоимость и обслуживание.

Для текущего MVP лучше сначала искать API/HTML/SSR/sitemap методы.

10. fetchWithTimeout

Файл: `src/data/sources/http.ts`

Все scraper-like источники используют `fetchWithTimeout`.

Зачем:

- сайты могут зависнуть;
- один плохой источник не должен подвесить Explore;
- unified search запускает много источников параллельно;
- timeout превращает вечное ожидание в обычную ошибку, которую query слой может пропустить.

Принцип:

```

■■■■■■■■ AbortController
setTimeout -> abort
fetch(url, { signal })
finally clearTimeout

```

Важно: timeout/error не должны записывать источник как "dead chapters". Dead cache можно писать только после успешного ответа с пустым списком глав.

11. Query layer: `src/data/queries.ts`

Этот файл - мост между UI и источниками/базой.

Основные hooks:

- `useSourcesQuery`
- `useTrending`
- `useSearch`
- `useUnifiedSearch`
- `useMatches`
- `useReadableFallback`
- `useMangaDetails`
- `useChapters`
- `useDeadChapters`
- `useChapterPages`
- `useContinueReading`
- `useLibrary`
- `useUpdates`
- `useReadChapters`
- `useWorkPref`
- `useReadChapterNumbers`

- mutations для library/favorite/status/read.

`useTrending`

Вызывает:

```
SourceManager.require(sourceId).trending({ lang, sort, limit })
```

Используется Home и Top.

`useSearch`

Поиск по одному активному источнику.

`useUnifiedSearch`

Поиск по всем enabled источникам.

Алгоритм:

1. взять все providers;
2. оставить только `supportsSearch`;
3. проверить `isSourceUsable`: язык включен и источник не hidden;
4. запустить `p.search(query)` параллельно;
5. если источник упал - вернуть пустой массив;
6. объединить все результаты через `clusterSearchResults`.

Это хорошая архитектура: один плохой сайт не убивает весь поиск.

`useMangaDetails`

Получает детали и сохраняет базовую карточку в SQLite через `cacheManga`.

Зачем cacheManga:

- Library и Continue Reading должны работать быстро;
- если сайт временно умер, локально остается title/cover.

`useChapters`

Получает главы и записывает результат в `dead\_chapters`:

- если глав > 0 - очищает dead cache;
- если глав = 0 - запоминает, что этот source/title/lang пустой.

Это защищает Explore и "Read on" от источников с нулем глав.

`useReadableFallback`

Если текущий источник открылся, но глав нет, app пробует другие variants этого же work и ищет первый, где главы есть.

Это как спасательный мост:

```
MangaDex title -> 0 readable chapters  
variants include Mangapill or MangaLib  
fallback finds source with chapters  
UI shows "Read on X"
```

`useUpdates`

Для тайтлов из библиотеки, которые пользователь начал читать:

1. взять library;
2. оставить started;
3. загрузить chapters через query cache;
4. сравнить с read chapter numbers/progress;
5. посчитать unread;
6. показать список обновлений.

12. Local SQLite: что хранится на устройстве

Файл: `src/data/local/db.ts`

База называется `mangaapp.db`.

Таблицы:

`cached\_manga`

Кэш title/cover/description:

- `source\_id`
- `external\_id`
- `title`
- `cover\_url`
- `description`
- `updated\_at`

Нужна для Library и Continue Reading.

`reading\_progress`

Прогресс чтения:

- `source\_id`
- `manga\_external\_id`

- `chapter\_id`
- `chapter\_number`
- `language`
- `page\_index`
- `percent`
- `updated\_at`
- `dirty\_for\_sync`
- `read`

`read` становится 1, когда пользователь дочитал почти до конца или вручную отметил главу прочитанной.

`library\_items`

Библиотека:

- `source\_id`
- `manga\_external\_id`
- `status`
- `favorite`
- `last\_read\_at`
- `dirty\_for\_sync`

Status может быть:

- reading
- plan
- completed
- on_hold
- dropped

`work\_source`

Связи одного тайтла между источниками.

Например:

```
group_id = g_xxx
MangaDex:solo-leveling
Mangapill:12345
MangaLib:solo-leveling
```

Зачем:

- favorite/status/library применяются ко всему work;
- read numbers можно переносить между источниками;
- "Also available on" становится не просто UI, а сохраняемой связью.

``work_pref``

Предпочтительный источник и язык для work.

Если пользователь для тайтла выбрал ReManga/RU или MangaDex/EN, app может открыть именно его в следующий раз.

``dead_chapters``

Запоминает source/title/language, где успешно проверили главы и получили 0.

Важно:

- TTL 5 дней;
- само лечится;
- не пишется при ошибке сети;
- используется Explore и Manga page для скрытия пустых источников.

13. Как работает Reader

Файл: ``app/reader/[chapterId].tsx``

Reader получает params:

- ``chapterId``
- ``sourceId``
- ``mangaId``
- ``chapterNumber``
- ``lang``
- ``startPage``

Потом:

1. ``useChapterPages(sourceId, chapterId)`` получает страницы.
2. ``useChapters(sourceId, mangaId, lang)`` получает список глав для prev/next.
3. Рендер страниц идет через FlashList в vertical mode или FlatList в paged mode.
4. Expo Image загружает картинки с cache.
5. При смене видимой страницы сохраняется progress через debounce.
6. Если percent ≥ 0.9 , глава помечается read.
7. Next chapter prefetch делает следующий переход быстрее.
8. Gesture layer дает pinch zoom, pan и double-tap.
9. Settings sheet управляет режимом чтения, направлением, gap, brightness, keepAwake.

Почему ``lang`` фиксируется при открытии Reader:

MangaDex имеет разные chapter id на разные языки. Если пользователь начал читать EN, а global language переключился на RU, prev/next не должны внезапно смешать языки. Поэтому Reader lock-ит language из params.

14. Как работают языки

Есть несколько понятий:

``enabledLanguages``

Хранится в ``settings.store.ts``. По умолчанию:

```
['en', 'ru']
```

Это языки контента, которые пользователь хочет видеть.

``language``

Текущий выбранный язык. Например, ``en`` или ``ru``.

``source.languages``

Языки, которые provider поддерживает.

Например:

- MangaDex: много языков.
- Mangapill: en.
- MangaLib: ru.
- ReManga: ru.
- WEBTOON: en.

Sources screen

Sources показывает:

```
■■■■■■ ■■■■ -> ■■■■■■ ■■■■■■■■■■ ■■■■■ ■■■■■ -> ■■■■■■ source
```

Explore unified search

Unified search берет только источники, которые:

- не hidden;
- имеют хотя бы один язык из `enabledLanguages`.

Manga page language chips

Если текущий source поддерживает несколько включенных языков, показываются language chips.

15. Also available on / Read on

Файлы:

- `src/data/sources/match.ts`
- `app/manga/[id].tsx`
- `src/data/local/db.ts`

Идея: найти этот же work на других источниках.

Алгоритм упрощенно:

1. Берем текущий `MangaDetails`.
2. Собираем title и altTitles.
3. Для каждого другого provider:
 - выбираем подходящие title queries по языку;
 - вызываем `search`;
 - считаем title score;
 - для лучших кандидатов вызываем `getMangaDetails`;
 - уточняем score через metadata: год, авторы, жанры;
 - возвращаем exact или probable match.
4. Manga page накапливает variants.
5. `linkWork` сохраняет variants в SQLite.
6. UI показывает `Read on`.

Защиты от неправильного matching:

- one-word title должен совпасть exact;
- variant markers должны совпадать: novel, spin-off, colored, sequel и т.п.;
- score thresholds;
- metadata penalties;
- не объединяются два результата из одного source в один cluster.

Проблема, которую надо помнить: matching не может быть идеальным. Источники часто называют тайтлы по-разному, а иногда похожие названия - разные работы. Поэтому `probable` лучше показывать осторожно.

16. Почему иногда 0 глав

Причины:

- тайтл есть в каталоге, но главы licensed/external;
- выбран не тот язык;
- источник показывает metadata, но reading закрыт;
- сайт поменял HTML;

- парсер не нашел главы;
- главы платные;
- геоблок;
- "Also available" нашел похожий, но не тот тайтл.

Что уже есть:

- MangaDex search/trending фильтрует `hasAvailableChapters=true`.
- Manga page показывает fallback, если активный источник пустой.
- `dead\_chapters` запоминает успешные пустые проверки.
- Explore фильтрует known-dead variants.

Что можно улучшать дальше:

- добавить `chapterCount` или `hasReadableChapters` в `MangaSearchResult`, когда source знает count дешево;
- не делать тяжелый `getChapters` на каждую карточку поиска;
- source-level фильтры в providers;
- точнее различать `empty`, `blocked`, `paid`, `parser\_broken`;
- Diagnostics может показывать "search ok/details ok/chapters empty/pages ok".

17. Как добавить новый источник

Шаги:

1. Создать файл:

```
src/data/sources/providers/newsource.ts
```

2. Реализовать класс:

```
export class NewSourceProvider implements SourceProvider {
  id = 'newsource';
  name = 'New Source';
  languages = ['en'];
  type = 'scraper' as const;
  supportsSearch = true;
  supportsReading = true;

  async trending(options?: SearchOptions) { ... }
  async search(query: string, options?: SearchOptions) { ... }
  async getMangaDetails(externalId: string) { ... }
  async getChapters(externalId: string, lang?: string) { ... }
  async getChapterPages(chapterId: string) { ... }
}
```

3. Добавить import и `new NewSourceProvider()` в `registry.ts`.

4. Добавить имя/цвет в `sourceMeta.ts`.

5. Проверить:

- search;
- details;

- chapters;
- pages;
- images with headers;
- language;
- Diagnostics;
- zero chapters behavior.

18. Как исследовать сайт перед добавлением

Лучший порядок:

1. Проверить homepage:

- status 200?
- Cloudflare/DDoS challenge?
- HTML server-rendered или пустой JS shell?

2. Проверить search:

- есть API?
- есть HTML result page?
- есть sitemap?
- есть Next/Nuxt payload?

3. Проверить manga details:

- title?
- cover?
- description?
- genres/authors/status?

4. Проверить chapters:

- список глав в HTML/API?
- есть paid/locked главы?
- есть language?
- есть branch/team?

5. Проверить pages:

- картинки прямо в HTML?
- JSON endpoint?
- script array?
- нужны Referer/Cookie/User-Agent?
- ссылки временные?

6. Проверить Android:

- картинки грузятся через Expo Image?
- headers передаются?
- нет huge memory usage?

19. Оценка методов добычи данных

Самые лучшие:

- официальный JSON API;
- стабильный публичный JSON endpoint;
- SSR data в HTML;
- sitemap + server-rendered details;
- простые HTML pages без challenge.

Средние:

- HTML scraping с регулярками;
- Next/Nuxt payload reverse;
- GraphQL reverse без introspection.

Плохие для MVP:

- Cloudflare challenge;
- DDoS-Guard challenge;
- headless browser;
- WebView extractor;
- FlareSolvrr/proxy;
- geo-blocked chapters.

20. Backend в проекте

Папка `backend/` все еще есть. Она содержит старую/альтернативную архитектуру:

- `backend/src/server.ts`
- `backend/src/sources/\*`
- `backend/src/services/cache.ts`
- `backend/src/services/health.ts`

Идея backend-версии:

app -> backend normalized API -> providers -> websites

Текущая app-версия:

app -> on-device providers -> websites

Почему on-device сейчас полезен:

- RU сайты видят residential/mobile IP пользователя, а не datacenter IP сервера;
- нет расходов на сервер;
- меньше backend обслуживания;
- проще для Android MVP.

Минусы on-device:

- логика scraping уезжает в app build;
- если сайт поменялся, нужен новый build;
- некоторые сайты с Cloudflare все равно блочат;
- unified search грузит телефон и сеть пользователя.

Гибридный путь на будущее:

- оставить простые источники on-device;
- тяжелые Cloudflare/geo источники через проху/backend;
- добавить allowlist, rate limit, cache;
- не проксировать все подряд.

21. Почему "движки, а не сайты" - хорошая идея

Сейчас каждый provider написан под конкретный сайт. Это нормально для MVP.

Но когда появится 2-3 сайта одной структуры, лучше вынести engine:

- `MadaraProvider`
- `GroupleProvider`
- `NextPayloadProvider`
- `NuxtPayloadProvider`
- `SitemapHtmlProvider`
- `WordPressMangaProvider`

Тогда новый сайт становится конфигом:

```
new MadaraProvider({
  id: 'site1',
  name: 'Site 1',
  baseUrl: 'https://site1.com',
  languages: ['en'],
})
```

Правильный порядок:

1. Сделать один сайт руками.
2. Сделать второй похожий.
3. Только потом выносить engine.

Не надо строить огромный universal engine заранее. Он может оказаться ненужным.

22. Текущие сильные стороны приложения

- Хороший `SourceProvider` контракт.
- UI отделен от сайтов.
- On-device providers подходят для RU источников.
- TanStack Query правильно отделяет server/cache state от UI.
- SQLite хранит прогресс и библиотеку offline-first.
- Есть cross-source linking.
- Есть dead-chapters cache.
- Есть reader с вертикальным и paged mode.
- Есть source/language filters.
- Есть диагностика.
- Есть image headers для CDN.
- Есть timeout для медленных источников.

23. Текущие слабые места / что помнить

- README устарел и путает текущую архитектуру.
- Source health пока в `sourcesInfo` статический online, реальная проверка через Diagnostics.
- Scrapers хрупкие: HTML может меняться.
- Regex parsing быстро, но требует тестов.
- Matching "probable" может ошибаться.
- Некоторые строки в файлах выглядят с битой кодировкой в комментариях/лейблах, это стоит когда-нибудь почистить.
- New source = app update, если логика on-device.
- Cloudflare/DDoS-Guard не решаются обычным fetch.

24. Что изучать дальше, чтобы лучше делать такие apps

Тебе полезно прокачать:

- TypeScript interfaces/types.
- React hooks and state.
- TanStack Query mental model: queryKey, staleTime, invalidateQueries.
- SQLite basics: tables, primary key, indexes, migrations.
- HTTP basics: headers, Referer, User-Agent, redirects, status codes.
- Browser DevTools Network tab.
- HTML parsing.
- JSON API reverse engineering.

- Next/Nuxt SSR data.
- Rate limits and caching.
- Android safe areas and navigation.

25. Мини-шпаргалка: поток данных

Search all sources

Explore input

- > debounce 400ms
- > useUnifiedSearch(query)
- > SourceRegistry.all()
- > filter hidden/language
- > Promise.all(provider.search)
- > clusterSearchResults
- > filter dead_chapters
- > MangaCard grid

Open manga

MangaCard press

- > /manga/[id]?sourceId=...
- > useMangaDetails
- > cacheManga
- > useChapters
- > markChaptersChecked
- > useMatches
- > linkWork
- > display Read on / Language / Chapters

Read chapter

Chapter press

- > /reader/[chapterId]
- > useChapterPages
- > useChapters for prev/next
- > render pages
- > saveProgress debounce
- > if percent >= 0.9 mark read

Library

Add to Library

- > cacheManga
- > addToLibraryForGroup
- > SQLite library_items
- > useLibrary
- > Library screen

Updates

getLibrary

- > started titles only
- > fetch chapters
- > compare read chapter numbers
- > unread count
- > updates list

26. Как думать о будущем проекта

Если цель - хороший MVP/портфолио:

1. Не гнаться за сотнями сайтов.
2. Добавлять только источники, которые реально читаются.
3. Улучшить source health.
4. Сделать backup/export/import библиотеки.
5. Улучшить matching confirmations.
6. Добавить source capability labels: API, HTML, SSR, blocked, metadata-only.
7. Добавить tests/probes для providers.

Если цель - большой reader как Mihon/Tachiyomi:

1. Делать engines.
2. Отделить sources packages.
3. Добавить source updates вне app release.
4. Сделать health telemetry.
5. Сделать sync.
6. Осторожно думать про backend/proxy только для сложных источников.

27. Самое главное в одном абзаце

Твое приложение уже построено вокруг правильной идеи: "UI не знает сайты, сайты спрятаны за SourceProvider". Данные с сайтов берутся либо через JSON API, либо через HTML scraping, либо через sitemap/mobile endpoints, после чего приводятся к единым типам `MangaSearchResult`, `MangaDetails`, `Chapter`, `ChapterPage`. TanStack Query управляет запросами и cache, SQLite хранит библиотеку/прогресс/связи между источниками, Zustand хранит настройки пользователя, а Expo Router связывает экраны. Чтобы добавлять новые сайты в будущем, тебе не надо переписывать app - надо изучить сайт, понять лучший способ добычи данных, написать provider, зарегистрировать его и проверить search/details/chapters/pages.